SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

---

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparis on Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentat ion	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Compare Sequential File Organization and Heap File Organization in terms of Data Insertion, Deletion, and Retrieval Efficiency.

Answer:

Sequential file organization stores records in a particular sequence, usually according to the value of a search key such as Student ID or Employee ID. In contrast, **heap file organization** stores records without any particular ordering, generally placing a new record wherever free space is available.

Aspect	**Sequential File Organization**	**Heap File Organization**
Record arrangement	Records are stored in sorted/key order	Records are stored without a specific order
Insertion	Relatively slower because correct position must be maintained	Very fast because records can be placed in available space
Deletion	More complex because sequence/order must be maintained	Easier; record can simply be marked/deleted
Sequential retrieval	Very efficient	Less efficient because records are unordered
Search by key	Efficient when searching according to ordering key	Usually requires linear search without an index
Updating records	May require movement if size/order changes	Generally simpler
Best suited for	Applications requiring ordered or sequential access	Applications with frequent insertions

Insertion

In a sequential file, a new record must generally be inserted in its correct position. For example, if employee records are maintained in increasing order of Employee_ID, inserting Employee 105 between 104 and 106 may require shifting records or managing overflow blocks. In a heap file, the new record can normally be placed in any available location. Therefore, insertion is comparatively faster.

Deletion

Deletion in sequential organization can be more complicated because removing a record may create a gap and disturb the physical organization. Some systems therefore use deletion markers and perform periodic reorganization.

In a heap file, a record can be deleted directly, and the resulting free space can later be reused.

Retrieval

Sequential organization provides efficient **ordered and sequential access**. For example, generating a report of all employees ordered by employee ID can be efficient.

Heap organization is better when the system frequently performs insertions but does not require records to be physically ordered. However, searching for a particular record without an index may require scanning many blocks.

Critical Analysis

Therefore, **heap organization prioritizes fast insertion and simple modification**, whereas **sequential organization prioritizes ordered retrieval and sequential processing**.

Example:

A temporary transaction table with frequent insertions may benefit from heap organization, while a payroll file that is frequently processed in employee-ID order may benefit from sequential organization.

Conclusion

Neither organization is universally superior. **Heap files are preferable for write-heavy workloads**, while **sequential files are more suitable when ordered or sequential retrieval is important**.

Q2. Differentiate Between Clustered Indexing and Non-Clustered Indexing with Suitable Database Examples.

Answer:

Clustered indexing determines the physical order of records in a table, whereas a **non-clustered index** maintains a separate index structure containing key values and references to the actual records.

Aspect	**Clustered Indexing**	**Non-Clustered Indexing**
Physical ordering	Records are physically organized according to index key	Records remain physically independent of index
Number generally possible	Usually one clustered ordering per table	Multiple non-clustered indexes can exist
Storage	Closely associated with actual data organization	Requires separate index structure
Range queries	Highly efficient	Efficient, but may require additional record lookups
Exact search	Efficient	Efficient
Insert/update cost	Can be higher because physical order may need maintenance	Index must be updated
Best use	Range and ordered retrieval	Multiple search conditions

Clustered Index

- Suppose a student table is organized according to:
  Student_ID
- If the table has a clustered index on Student_ID, records are stored in an order corresponding to that index.

For example:

101 → Arun

102 → Bala

103 → Charan

104 → David

105 → Kavin

Searching for students with IDs between 102 and 104 can be efficient because related records are physically close to one another.

Non-Clustered Index

- A non-clustered index is maintained separately from the table data.
- For example, the Student table may physically be organized according to Student_ID, while a non-clustered index is created on:
- Department

The index might contain:

- CSE → references to Student records
- ECE → references to Student records
- MECH → references to Student records

The database uses the index to find matching records and then follows the references to the actual data.

Critical Analysis

Clustered indexing is particularly beneficial for **range queries and ordered retrieval**, because related records are stored close together.

Non-clustered indexing provides flexibility because multiple indexes can be created on different columns. However, every additional index consumes storage and must be maintained when records are inserted, deleted, or updated.

Example

For an employee table:

Employee(Employee_ID, Name, Department, Salary)

A clustered index on Employee_ID is useful for retrieving employees in ID order.

A non-clustered index on Department is useful for queries such as:

```
mysql> SELECT * FROM Employee
    -> WHERE Department = 'CSE';
```

Conclusion

Clustered indexing is better when physical ordering and range retrieval are important, while non-clustered indexing is better when multiple alternative search paths are required.

Q3. Compare Primary Indexing and Secondary Indexing Based on Storage Requirements and Search Performance.

Answer:

A **primary index** is generally associated with a file whose records are physically ordered according to the primary/search key. A **secondary index** provides an additional access path on a field that is not the physical ordering field.

Aspect	**Primary Index**	**Secondary Index**
Based on	Ordering/search key	Non-ordering attribute
Physical ordering	Data file is ordered according to index key	Data file need not be ordered according to indexed field
Storage requirement	Usually lower, especially for sparse primary indexes	Usually higher because more index entries may be required
Search performance	Very efficient on primary/search key	Efficient for searches on indexed secondary attributes
Number of indexes	Generally one primary ordering	Multiple secondary indexes possible
Duplicate values	Usually key values are unique	Duplicate values may exist
Maintenance	Relatively simpler	Can be more expensive with frequent updates

Storage Requirement

A primary index can often be **sparse**, meaning one index entry can represent a block rather than every record.

For example:

Index:

100 → Block 1

200 → Block 2

300 → Block 3

The database can locate the appropriate block and then search within it.

A secondary index is often **dense**, particularly when the indexed attribute does not determine the physical ordering of records.

For example:

CSE → Record references

ECE → Record references

MECH → Record references

Therefore, secondary indexing can require additional storage.

Search Performance

If a table is physically ordered by Student_ID, a primary index on Student_ID can quickly identify the required block.

For example:

```
mysql> SELECT * FROM Student
    -> WHERE Student_ID = 105;
```

The primary index can locate the relevant block efficiently.

A secondary index is useful for attributes such as:

```
mysql> SELECT * FROM Employee
    -> WHERE Department = 'CSE';
```

Even if the table is physically ordered by Student_ID, the secondary index provides a separate search path.

Critical Analysis

The major advantage of primary indexing is **efficient access with relatively lower index storage**, especially when a sparse index is possible.

The major advantage of secondary indexing is **flexibility**. It allows users to search efficiently using attributes other than the physical ordering key.

However, secondary indexes increase storage consumption and maintenance overhead.

Conclusion

Primary indexing provides efficient access through the main ordering key with relatively lower storage overhead, whereas secondary indexing improves flexibility by providing additional search paths at the cost of extra storage and maintenance.

Q4. Analyze the Differences Between Static Hashing and Dynamic Hashing in Database Systems.

Answer:

Hashing uses a hash function to map a search key to a bucket where the corresponding record is stored.

The two major approaches are **static hashing** and **dynamic hashing**.

Aspect	**Static Hashing**	**Dynamic Hashing**
Number of buckets	Usually fixed	Can grow or shrink
Database growth	Less flexible	Highly flexible
Collision handling	Overflow buckets/chaining commonly required	Buckets can split
Performance with growth	May degrade	Generally remains more stable
Storage management	Simple initially	More complex
Scalability	Limited	High
Best suited for	Relatively stable datasets	Frequently growing/shrinking datasets

Static Hashing

In static hashing, the number of buckets is determined when the hash structure is created.

For example:

h(key) = key MOD 5

This produces five primary buckets:

0

1

2

3

4

If the database grows significantly, many records may map to the same bucket. This creates **overflow buckets**.

Dynamic Hashing

- Dynamic hashing allows the hash structure to change as the database grows.
- When a bucket becomes full, the database can split the bucket and redistribute records.
- This reduces the dependence on long overflow chains.

Performance Analysis

- Suppose a database initially contains 1,000 records and later grows to 1,000,000 records.
- A static hash structure designed for the original size may develop significant overflow chains. As a result, exact-match searches may require additional disk accesses.
- Dynamic hashing can expand its structure as the number of records increases, maintaining relatively efficient access.

Example

An e-commerce database may grow continuously:

1 lakh customers

↓

5 lakh customers

↓

20 lakh customers

Dynamic hashing is more suitable because the structure can adapt to growth.

Critical Analysis

- Static hashing is **simple and predictable**, but its performance can deteriorate when the database size changes significantly.
- Dynamic hashing requires more complex bucket management but provides **better scalability and adaptability**.

Conclusion

Static hashing is appropriate for relatively stable datasets, while dynamic hashing is more suitable for large and continuously changing databases.

Q5. Compare B-Tree Indexing and B+ Tree Indexing for Large-Scale Database Applications.

Answer:

Both **B-Tree and B+ Tree** are balanced tree-based indexing structures used to provide efficient searching, insertion, and deletion.

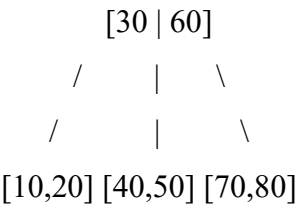
The major difference is how they store data pointers and organize leaf nodes.

Aspect	**B-Tree**	**B+ Tree**
Data storage	Data/record pointers may occur in internal and leaf nodes	Data/record pointers are stored at leaf level
Internal nodes	Can contain keys and data pointers	Generally contain keys and child pointers
Leaf nodes	May not form a sequential linked structure	Usually linked sequentially
Range queries	Efficient	Very efficient
Fan-out	Comparatively lower	Generally higher
Tree height	May be slightly greater	Often lower
Sequential access	Less convenient	Highly efficient
Database usage	Used in some indexing systems	Very common in database indexes

B-Tree

A B-Tree distributes keys and pointers throughout the tree.

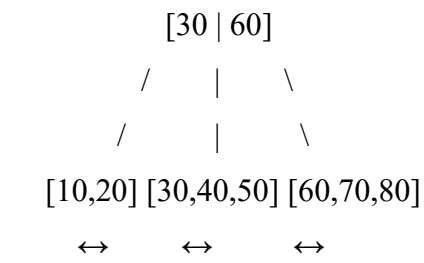
Conceptually:



Data references may exist at different levels.

B+ Tree

In a B+ Tree, internal nodes primarily guide the search, while actual record pointers are maintained at the leaf level.



The leaf nodes are linked, making sequential and range access efficient.

Large-Scale Database Analysis

For a query such as:

```
mysql> SELECT * FROM Employee
    -> WHERE Salary BETWEEN 50000 AND 80000;
```

A B+ Tree can locate the first relevant leaf and then traverse linked leaf nodes until the required range is covered.

This makes B+ Trees particularly suitable for **range queries**.

B+ Trees also have high fan-out because internal nodes generally contain keys and child pointers rather than complete data records. Higher fan-out means fewer tree levels and potentially fewer disk accesses.

Critical Analysis

B-Trees can support efficient direct searches, but B+ Trees provide advantages for database workloads involving **large datasets, range queries, and sequential processing**. Therefore, B+ Trees are generally preferred in large-scale database indexing.

Conclusion

B-Tree and B+ Tree both provide balanced logarithmic search performance, but B+ Tree is generally more suitable for database systems because of its high fan-out, linked leaves, and efficient range/sequential access.

Q6. Differentiate Linear Hashing and Extendible Hashing in Terms of Bucket Management and Scalability.

Answer:

Linear hashing and **extendible hashing** are dynamic hashing techniques that allow a hash structure to expand as the number of records increases.

Aspect	**Linear Hashing**	**Extendible Hashing**
Expansion	Buckets split progressively	Buckets split according to directory/hash bits
Directory	Does not require a directory	Uses a directory
Bucket splitting	Controlled by a split pointer	Triggered when a bucket overflows
Growth	Gradual/linear	Can expand directory, often by doubling
Memory overhead	Lower because no directory is required	Higher due to directory
Scalability	Good	Very good
Management complexity	Relatively simpler	More complex

Linear Hashing

Linear hashing maintains a **split pointer**.

When the structure grows, buckets are split in a predetermined sequence rather than randomly.

For example:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

↓

Split Bucket 0

↓

Split Bucket 1

↓

Split Bucket 2

This gradual expansion avoids requiring a complete restructuring of the hash table.

Extendible Hashing

Extendible hashing uses a **directory** containing pointers to buckets.

The directory uses hash-value bits to identify the appropriate bucket.

When a bucket overflows:

1. The bucket is split.
2. If necessary, the directory size is increased.
3. Records are redistributed according to additional hash bits.

Scalability Analysis

Linear hashing provides smooth growth without requiring a large directory. It is useful when gradual expansion is preferred.

Extendible hashing can provide very fast bucket location because the directory directly identifies the relevant bucket. However, the directory can consume additional memory.

Critical Analysis

The key difference is the **method of expansion**:

- Linear hashing → gradual bucket splitting.
- Extendible hashing → directory-based bucket splitting.

Conclusion

Linear hashing offers simpler and gradual scalability with lower directory overhead, while extendible hashing provides more direct bucket management and excellent scalability at the cost of directory storage and management complexity.

Q7. Compare Dense Indexing and Sparse Indexing with Respect to Memory Usage and Access Speed.

Answer:

A **dense index** contains an index entry for every search-key value or record, while a **sparse index** contains entries only for selected search-key values, typically one entry per data block.

Aspect	**Dense Index**	**Sparse Index**
Index entries	More entries	Fewer entries
Memory usage	Higher	Lower
Search speed	Usually faster/direct	Requires additional search within block
Storage overhead	High	Low
Maintenance	More expensive	Comparatively simpler
Applicability	Useful for secondary indexes	Common with ordered files
Range retrieval	Efficient	Efficient when data is ordered

Dense Index

Consider:

Records:

101

102

103

104

105

A dense index may contain:

101 → Record 101

102 → Record 102

103 → Record 103

104 → Record 104

105 → Record 105

Every record has an index entry.

This provides fast access but consumes more storage.

Sparse Index

A sparse index may contain:

101 → Block 1

104 → Block 2

107 → Block 3

The index identifies the relevant block, after which the database searches within that block.

Memory Usage

If a table contains millions of records, a dense index can become very large because it contains many entries.

A sparse index can significantly reduce memory and storage requirements.

Access Speed

Dense indexing can provide faster direct access because the exact record or key is represented in the index.

Sparse indexing may require an additional search within the identified data block.

However, sparse indexes can still be highly efficient because they are much smaller and can potentially fit more effectively in memory.

Critical Analysis

The choice is therefore a **trade-off between storage and speed**.

Dense Index

More memory → Faster direct access

Sparse Index

Less memory → Additional block-level search

Conclusion

Dense indexing is preferred when faster access is more important than index storage, whereas sparse indexing is preferred when reducing memory/storage overhead is important and the underlying data is appropriately ordered.

Q8. Analyze the Advantages and Limitations of Indexed Sequential File Organization Versus Direct File Organization.

Answer:

Indexed Sequential File Organization (ISFO) stores records sequentially according to a key and maintains an index for faster access.

Direct File Organization uses a direct addressing or hashing technique to locate records without requiring sequential traversal.

Aspect	**Indexed Sequential File Organization**	**Direct File Organization**
Organization	Records stored sequentially with index	Records located directly using address/hash
Exact search	Efficient	Very efficient
Range search	Excellent	Generally less suitable
Sequential processing	Excellent	Poorer
Insertion	May require overflow handling	Generally efficient
Complexity	Moderate	Relatively simple for exact access
Best suited for	Sequential + random access	Exact-match/random access

Advantages of Indexed Sequential Organization

1. Supports both sequential and direct access

It can support:

Sequential access

+

Indexed direct access

This makes it flexible.

2. Efficient range queries

Suppose we need:

- WHERE Employee_ID BETWEEN 1000 AND 1100

Because records are ordered, the database can locate the beginning of the range and process subsequent records efficiently.

3. Faster search

The index reduces the number of blocks that need to be examined.

Limitations

- Maintaining the index creates additional overhead.
- Insertions may create overflow areas.
- Periodic reorganization may be required.
- Storage is required for both data and index structures.

Direct File Organization

Direct organization maps a search key to a storage location, commonly through hashing.

For example:

Hash(Employee_ID) → Bucket

Advantages

1. Very fast exact-match access

For:

- WHERE Employee_ID = 1050

hashing can directly identify the appropriate bucket.

2. Efficient random access

Records do not need to be scanned sequentially.

Limitations

Direct organization is not ideal for range queries.

For example:

- WHERE Employee_ID BETWEEN 1000 AND 1100

Hashing does not preserve key ordering, so retrieving a continuous range can require examining many buckets.

Critical Analysis

The major distinction is:

- **Indexed sequential organization = balanced support for sequential + random access**
- **Direct organization = optimized for direct/exact access**

Conclusion

Indexed sequential organization is preferable when an application needs **both sequential processing and range retrieval**, while direct organization is better when **fast exact-match access** is the dominant requirement.

Q9. Compare Single-Level Indexing and Multi-Level Indexing for Efficient Database Retrieval Operations.

Answer:

Single-level indexing uses one index structure between the search key and the data file. **Multi-level indexing** creates additional index levels over the original index when the index itself becomes large.

Aspect	Single-Level Indexing	Multi-Level Indexing
Number of levels	One	Multiple
Index size	Smaller for small datasets	Handles very large indexes
Search process	Search index → data	Search top index → lower index → data
Disk access	Can increase when index becomes large	Reduces search space at each level
Scalability	Limited for huge datasets	Highly scalable
Complexity	Simple	More complex
Large databases	Less suitable	Highly suitable

Single-Level Indexing

Suppose the database has:

Index

↓

Data File

- The index directly points to the relevant data block.
- For a small database, this can be highly efficient.
- However, if the index itself becomes very large, searching it can require multiple disk accesses.

Multi-Level Indexing

Multi-level indexing creates an index on the first-level index:

Level 2 Index

↓

Level 1 Index

↓

Data File

For a very large database:

Top-level index

↓

Second-level index

↓

First-level index

↓

Data blocks

Each level reduces the number of entries that must be examined.

Example

- Suppose an index contains 1,000,000 entries.
- Searching the entire index would be expensive.

Instead, a higher-level index may point to groups of lower-level index blocks:

Level 2 → identifies index block

Level 1 → identifies data block

Data → retrieves record

This hierarchical structure reduces the number of disk accesses.

Critical Analysis

Single-level indexing is simpler and has lower structural overhead, but its effectiveness decreases as the index becomes very large.

Multi-level indexing introduces additional levels but significantly improves scalability.

This concept forms the basis of structures such as **B-Trees and B+ Trees**.

Conclusion

Single-level indexing is appropriate for smaller indexes, whereas multi-level indexing is more efficient and scalable for large databases because it reduces the number of entries searched at each stage.

Q10. Evaluate the Performance of Hashing Techniques and Indexing Techniques for Exact-Match and Range Queries in Databases.

Answer:

Hashing and indexing are both important techniques for improving database retrieval performance. However, they behave differently depending on the type of query.

The two major query categories are:

1. **Exact-match queries**
2. **Range queries**

Comparison

Query / Feature	Hashing	B-Tree / B+ Tree Indexing
Exact-match search	Excellent	Excellent
Range search	Poor	Excellent
Ordered retrieval	Poor	Excellent
Average equality lookup	Very fast	Very fast
Sequential access	Not naturally supported	Well supported
Data ordering	Not maintained	Maintained
Collision/tree maintenance	Hash collisions must be handled	Tree balancing required
Best use	Equality conditions	Equality + range conditions

1. Performance for Exact-Match Queries

Consider:

```
mysql> SELECT * FROM Student
    -> WHERE Student_ID = 105;
```

- A hashing technique can apply:
- Hash(105) → Bucket
- The system can directly identify the bucket containing the record.
- Therefore, hashing is highly effective for exact-match queries.

Indexing

- A B+ Tree index can also efficiently locate the exact value.

Conceptually:

Root

↓

Internal Node

↓

Leaf Node

↓

Record

- The search follows a path from the root to the relevant leaf.
- Thus, both techniques provide efficient exact-match retrieval.

2. Performance for Range Queries

Consider:

```
mysql> SELECT * FROM Employee
    -> WHERE Salary BETWEEN 50000 AND 80000;
```

Hashing

Hashing is not suitable for this type of query because hashing deliberately distributes keys across buckets.

For example:

50000 → Bucket 4

50001 → Bucket 1

50002 → Bucket 8

50003 → Bucket 2

Numerically close values may be stored in completely different buckets.

Therefore, the database cannot simply scan neighboring buckets to obtain the required range.

B+ Tree Indexing

A B+ Tree maintains keys in sorted order at the leaf level.

For example:

50000 ↔ 55000 ↔ 60000 ↔ 65000 ↔ 70000 ↔ 75000 ↔ 80000

The database can:

1. Locate 50,000.
2. Traverse the linked leaf nodes.
3. Stop after 80,000.

Therefore, B+ Tree indexing is highly effective for range queries.

3. Exact-Match Performance Analysis

For an exact query:

```
mysql> WHERE Student_ID = 10025
```

Hashing can be preferable because the hash function can directly determine the target bucket.

However, a B+ Tree can also provide excellent performance, especially when the index is already available and maintained.

4. Range Query Performance Analysis

For:

```
-> WHERE Student_ID BETWEEN 10000 AND 11000
```

B+ Tree indexing is clearly preferable because the indexed values are ordered.

Hashing does not preserve ordering, making range retrieval inefficient.

5. Storage and Maintenance

Hashing requires management of:

- Buckets
- Collisions
- Overflow
- Bucket expansion in dynamic hashing

Indexing requires management of:

- Index nodes
- Tree balancing
- Insertions
- Deletions
- Splitting/merging nodes

Therefore, both have maintenance costs.

Overall Evaluation

The choice should depend on workload.

If the workload mainly contains:

```
    -> WHERE ID = 1001
    -> WHERE Account_No = 50025
    -> WHERE Roll_No = 1234
```

Hashing is highly suitable.

If the workload mainly contains:

```
  -> WHERE Salary BETWEEN 50000 AND 80000
  -> WHERE Date BETWEEN Jan 1 AND Jan 31
  -> ORDER BY Student_ID
```

B+ Tree indexing is more suitable.

Decision Table

Workload	Recommended Technique	Reason
Exact-match	Hashing	Direct bucket access
Exact-match + ordering	B+ Tree	Efficient search with ordered keys
Range query	B+ Tree	Maintains sorted keys
Sequential/range retrieval	B+ Tree	Linked ordered leaves
Frequently changing dataset	Dynamic hashing / B+ Tree	Adaptive structures
Equality-heavy workload	Hashing	Fast direct lookup

Final Evaluation

Hashing generally provides excellent performance for exact-match queries, whereas B+ Tree indexing provides more versatile performance for both exact-match and range queries.

Therefore, for a general-purpose large-scale database, **B+ Tree indexing is often the more flexible choice**, while **hashing is highly effective when the workload is dominated by equality searches**.

The correct choice ultimately depends on the application's **query pattern, data growth, update frequency, storage constraints, and requirement for range or ordered retrieval**.